EECS 182 Deep Neural Networks
Fall 2025 Anant Sahai and Gireeja Ranade

Optimization

These notes are a draft. There may be bugs and small errors, please report them to us if you find them. We will continue to improve them. These notes are based on lectures given by Gireeja Ranade and Anant Sahai in EECS182/282A at UC Berkeley in Fall 2025. Some parts of this document have been written by our fantastic TA Kevin Franz, and have been edited by the faculty. Other parts build on the cool notes generation script created by our student in the class, Jameson Liu. This script took in the lecture transcript and handwritten lecture notes to generate latex using an LLM-based pipeline. Other parts of this document are written by Gireeja Ranade and Anant Sahai. Thank you to all who have contributed, and please let us know if you find errors.

1 Introduction

Welcome to the notes for CS 282, Deep Learning at Berkeley. These notes are intended as a companion to the lectures and the homeworks. As usual, the most important thing for learning is to do the homework problems. The goal of these notes is to provide a reference for the key concepts covered during lecture, along with other useful information as we go along.

Prerequisites. To get the most out of this class, you should have a working knowledge of machine learning (CS189), linear algebra and optimization (EECS127), and probability (EECS126). You should also be comfortable with programming in Python and using libraries such as numpy and Pytorch. There are numerous great online materials for these above topics.

2 What is Deep Learning?

Within the umbrella of computer science, machine learning is unique in that it breaks our usual abstraction of programming. In ordinary cases, programs are written by a programmer, who will explicitly tell the computer what to do. However, over time it has become clear that there are certain tasks where we simply do not know how to write down a programmatic solution. For these reasons, an alternate paradigm was needed – one where computers could be programmed via *sets of examples*, rather than explicit instructions. As you may imagine, this departure would open up a whole new set of challenges – how to best represent and optimize arbitrary functions, how to generalize to new examples, and how to scalably implement these ideas. The study of these topics is, broadly, deep learning.

In this course, we will largely focus on the *core mechanics* of deep learning. This stands in contrast to a more application-focused curriculum, of which there are many great alternate resources. Rather, the main goal of this course is to equip the student with a sharp understanding of building blocks of deep learning systems, and the rationales behind *why* they work.

At the same time, it's important to acknowledge that deep learning is an empirical field. Unlike more formal branches of computer science, we often do not have the luxury of building up theories from axioms, and must instead attempt to reconcile our intuitions with experimental results. As a guiding principle, we will fall on the universal prior that *great ideas are simple*. When understood from the right angle, even complicated systems can be explained in terms of simple first principles. Throughout this course, we'll explore the

 All Rights Reserved. This may not be publicly shared without explicit permission.

core ideas in deep learning, with an eye towards seeing "past the noise" and into the underlying simplicity. Ideally, you will leave the course with a healthy skepticism that better strategies exist compared to those we use today.

3 Function Approximation and Neural Networks

The main goal of deep learning is to discover **functions** that accurately represent our data. The first step is to decide how we should represent an arbitrary functions, such that they can be iteratively adjusted to improve. For the simple case of a scalar-to-scalar function, a reasonable strategy is to use the **piecewise linear** representation. With enough pieces, we can approximate any continuous function, and there is a natural parameterization of these functions as a set of real numbers: x-locations, slopes, and starting height.

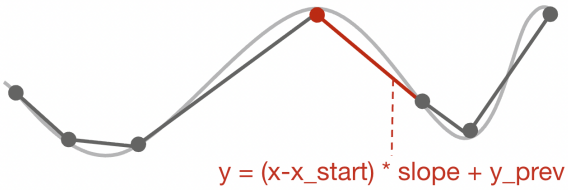


Figure 1: Piecewise Linear Function

The downside of this naive piecewise representation is that the piecewise parts need to be calculated sequentially, as each piece must start where the previous piece left off. As it turns out, there's a clever way to rewrite the piecewise linear function as a weighted sum of **elbow functions**:

$$y = \max(0, wx + b), \tag{1}$$

also known as the **rectified linear unit (ReLU)**. Writing a function in this way is nice because the pieces are *parallelizable* – we can compute all the elbow functions at once, and then combine them with a weighted sum later:

$$y = b' + \sum_i w'_i \cdot \max(0, w_i x + b_i). \tag{2}$$

This equation may start to look familar – it is in fact exactly a 2-layer **neural network**! Neural networks are a powerful function representation because of their inherent parallelizability. When considering a batch of inputs X, the above equation can be written as matrix-multiplications:

$$Y = b' + W' \cdot \max(0, WX + b), \tag{3}$$

in which the WX and $W'(\cdot)$ operations can be efficiently computed on a GPU.

The other reason neural networks are powerful is because of their heirarchical structure, which is where the "deep" in deep learning comes from. By stacking multiple layers of neural networks, we can create a **hierarchical representation** of our data. Earlier layers can create useful features, which are transformed by further layers, and so on. This hierarchical structure allows neural networks to learn complex functions – however, it will also contribute to the challenge of optimizing such networks, as we will see later on.

 All Rights Reserved. This may not be publicly shared without explicit permission.

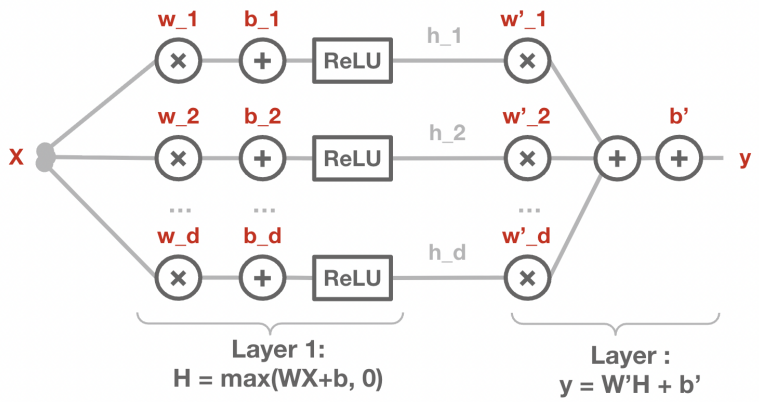


Figure 2: Block diagram of a 2-layer neural network. Each layer defines a piecewise linear composition of the previous layer. The ReLU activation is typically not applied after the last layer.

4 Training with Supervised Learning

Now that we have a way to represent functions, we would like to find a way to train such functions. Assuming we have a set of training data pairs (x_i, y_i), we additionally must define a **loss function** $\mathcal{L}(y, \hat{y})$ that measures how well our model's prediction $\hat{y}$ matches the true label y. A common choice for regression tasks is the **mean squared error (MSE)** loss:

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2. \tag{4}$$

For classification tasks, a common choice is the **cross-entropy** loss:

$$\mathcal{L}(y, \hat{y}) = -\sum_c y_c \log(\hat{y}_c), \tag{5}$$

where y_c is a one-hot encoding of the true class, and $\hat{y}_c$ is the predicted probability for class c.

Can we pick any loss function? In general, choosing a loss function is a design choice and there's no specific "right" answer, other than the requirement of differentiability. However, we generally like to stick with a few common choices. The MSE and cross-entropy losses are popular as they have concrete interpretations in terms of maximizing log-probability (of the Gaussian and categorical distributions, respectively), and they are convex in relation to the model's outputs.

The **empirical risk minimization** framework tells us to find the parameters θ of our model f_θ that minimize the average loss across our training data:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(y_i, f_\theta(x_i)). \tag{6}$$

To do so, we will rely on **gradient descent**, an iterative optimization algorithm that updates the parameters in the direction of steepest descent of the loss. The update rule for gradient descent is given by:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t), \tag{7}$$

where η is a scalar learning rate, and $\nabla_\theta \mathcal{L}(\theta_t)$ is the gradient of the loss with respect to the parameters. Gradient descent is the backbone of deep learning, and its strength comes from its universal applicability

 All Rights Reserved. This may not be publicly shared without explicit permission.

to network architectures. For an arbitrary neural network, we can compute the gradient of the loss with respect to the parameters using **backpropagation**, which is an efficient application of the chain rule. Today, backpropgation can be automatically handled using autodifferentiation libraries such as Pytorch.

5 Gradient Descent on Least-Squares Linear Regression

Gradient descent is not without its challenges. To understand when and how gradient descent converges, we will first study the simple case of **least-squares linear regression**, in which our parameters are a vector $w \in \mathbb{R}^d$, and our model is $f_w(X) = Xw$. The loss function is the MSE loss:

$$\mathcal{L}(W) = \|Xw - y\|^2, \tag{8}$$

where $X \in \mathbb{R}^{n \times d}$ is the matrix of input data, and $y \in \mathbb{R}^n$ is a vector of scalar outputs, resulting in a gradient with respect to w of:

$$\nabla_w \mathcal{L}(w) = 2X^\top (Xw - y). \tag{9}$$

For our specific case of least-squares linear regression, we can actually solve for the optimal solution in closed form. To do this, set the gradient to zero, and solve the resulting matrix relationship:

$$\nabla_w^* \mathcal{L}(w^*) = 2X^\top (Xw^* - y) = 0 \tag{10}$$

$$X^\top X w^* = X^\top y \tag{11}$$

$$w^* = (X^\top X)^{-1} X^\top y. \tag{12}$$

Now, let's examine how gradient descent will converge to this solution. To do this, let's write our iterates in terms of the **error term**, e.g. the delta from the optimal solution:

$$e_t = w_t - w^*. \tag{13}$$

Substituting into the update rule, we have:

$$e_{t+1} = w_{t+1} - w^* = w_t - 2\eta X^\top (Xw_t - y) - w^* \tag{14}$$

$$= w_t - 2\eta X^\top X w_t + 2\eta X^\top y - w^* \tag{15}$$

$$= w_t - 2\eta X^\top X w_t + 2\eta X^\top X w^* - w^* \tag{16}$$

$$= (I - 2\eta X^\top X)(w_t - w^*) \tag{17}$$

$$= (I - 2\eta X^\top X) e_t. \tag{18}$$

Look carefully, as we can now see a defining property of gradient descent. At each iteration, the error term is multiplied by the matrix $(I - 2\eta X^\top X)$. Thus, it is the **eigenvalues** of this matrix that determines whether the error term will shrink or grow over time, along each respective eigenvector.

Let's make this intuition a step clearer. Decomposing $X^\top X$ into its eigenbasis, we have:

$$X^\top X = V \Lambda V^\top, \tag{19}$$

where V is an orthogonal matrix of eigenvectors, and Λ is a diagonal matrix of eigenvalues λ_i. When we

 All Rights Reserved. This may not be publicly shared without explicit permission.

view the components of the error term in the eigenbasis $V^\top e_t$, we see the following relation:

$$e_{t+1} = (I - 2\eta V \Lambda V^\top) \tag{20}$$

$$e_t = V(I - 2\eta\Lambda)V^\top e_t. \tag{21}$$

$$V^\top e_{t+1} = (I - 2\eta\Lambda)V^\top e_t. \tag{22}$$

In other words, **along each eigenbasis direction, the error term decreases independently**, at a rate of $(1-2\eta\lambda_i)$. These components do not interact, and the total loss is simply the sum of the squared components, scaled by their respective eigenvalues.

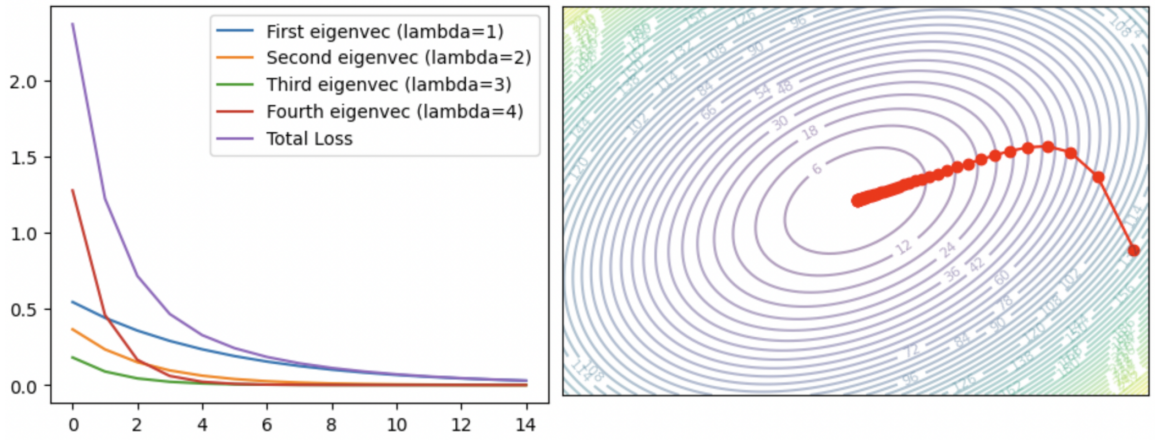


Figure 3: Left: Illustration of independent error decay along eigenbasis directions during gradient descent. Each component of the error vector decreases at a rate determined by the corresponding eigenvalue of $X^\top X$. **Right:** Visualization of two dimensions on the loss landscape. Optimization progress is bottlenecked by the slowest direction.

This analysis allows to interpet gradient descent as a *simultaneous descent* along many directions in parameter space. To ensure converge, it is neccessary that the largest eigenvalue of $(I - 2\eta X^\top X)$ is less than 1 in magnitude. This gives us a condition on the learning rate:

$$\eta < \frac{1}{\lambda_{\max}}. \tag{23}$$

What is the optimal learning rate? The amount of learning "progress" is bounded by the slowest moving direction, which is the direction with the smallest eigenvalue $\lambda_{\min}$. To maximize the rate of convergence, we want to pick a learning rate that maximizes descent along this direction, while still satisfying the bound above. This occurs when:

$$|1 - 2\eta\lambda_{\min}| = |1 - 2\eta\lambda_{\max}| \quad \rightarrow \quad \eta = \frac{2}{\lambda_{\min} + \lambda_{\max}}. \tag{24}$$

The ratio between the largest and smallest eigenvalues is known as the **condition number** of the problem, and is a measure of how "ill-conditioned" the problem is. Generally, the condition number is what limits the speed of convergence, and we will see later that many optimization techniques are motivated as ways to improve the condition number.

 All Rights Reserved. This may not be publicly shared without explicit permission.

6 Ridge Regularization

So far, we have assumed that there exists some "optimal" set of parameters, which is true from the perspective of empirical risk minimization, but may not be true in practice. In particular, if our training data is noisy, we may end up overfitting to this noise, leading to poor generalization. Additionally, our loss function may have *multiple* global minima, e.g. if $X^T X$ is not full rank. To address these issues, we can add a **regularization** term to our loss function, which penalizes large weights. A common choice is **ridge regularization** (also known as Tikhonov regularization), which adds an L_2 penalty on the weights:

$$\tilde{\mathcal{L}}(w) = \|Xw - y\|^2 + \gamma \|w\|^2, \tag{25}$$

where γ is a hyperparameter that controls the strength of the regularization. Gradient descent under the regularized objective results in an update rule with an additional "shrinkage" term:

$$w_{t+1} = \underbrace{(1 - 2\eta\gamma) w_t}_{\text{Shrinkage}} - \underbrace{2\eta X^\top (X w_t - y)}_{\text{Original gradient}}. \tag{26}$$

Instead of our original solution, we now have the following closed-form solution:

$$\tilde{w}^* = (X^\top X + \gamma I)^{-1} X^\top y. \tag{27}$$

Note that in this regularized case, $X^\top X + \gamma I$ is always full rank, and thus the solution is unique. This also means that we can opt to *transpose* the problem, and solve the **kernel ridge regression** problem instead:

$$\tilde{w}^* = X^\top (X X^\top + \gamma I)^{-1} y, \tag{28}$$

which is notably more efficient when $n << d$.

TODO: Relate the regularized solution to the original solution in terms of $(\lambda/(\lambda^2 + \gamma))$.

Regularization also has a clear affect on the eigenvalues of the gradient descent trajectory. Namely, each eigenvalue is increased by a constant value:

$$\tilde{\lambda}_i = \lambda_i + \gamma, \tag{29}$$

which notably has the effect of *improving the condition number* of the problem:

$$\frac{\lambda_{\max} + \gamma}{\lambda_{\min} + \gamma} < \frac{\lambda_{\max}}{\lambda_{\min}}. \tag{30}$$

This has the effect of making the eigenvalues more similar to each other, which in turn allows for a larger learning rate, and faster convergence. For deeper models, we will see an analogy to ridge regularization in the form of **weight decay**, a common technique in modern optimizers.

7 Stochastic Gradient Descent

In practice, it is expensive to compute the gradient over the entire dataset, especially for large datasets. Instead, we can use **stochastic gradient descent (SGD)**, which approximates the true gradient using a

 All Rights Reserved. This may not be publicly shared without explicit permission.

small batch of data points. The update rule for SGD is given by:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_\theta \mathcal{L}(y_i, f_\theta(x_i)), \tag{31}$$

where B is a random subset of the training data, known as a **mini-batch**. The size of the mini-batch is a hyperparameter that can be tuned, with larger batches generally leading to more stable updates, but also requiring more computation.

While inherently stochastic, the SGD update is equal to the true "full-batch" gradient in expectation, and a useful intution is that SGD follows the gradient descent trajectory, with some local noise at at each step. This noise can at times be beneficial – if the true gradient is stuck in a local minima, the noise from the stochastic batch can potentially escape this minima, and continue towards a better solution.

However, the noise is generally detrimental, and this is especially prevalent when we are close to the solution. In this setting, the noise from the stochastic gradient can cause the parameters to oscillate around the optimum rather than converging. For this reason, it is common to use a **learning rate schedule**, which decreases the learning rate over time, to allow for eventual convergence even in the presence of noise.

Does SGD converge? Yes, at least on the least-squares problem, and we can show it. Without loss of generality, let's consider the batchsize 1 case, in which we sample a *single* data point at follow the gradient of this point. Recall the dynamics of the error term from Equation 14 in the full-batch setting where $X \in \mathbb{R}^{n \times d}$:

$$e_{t+1} = (I - 2\eta X^\top X) e_t. \tag{32}$$

In the batchsize-1 case, we instead randomly sample a single data point $x_i \in \mathbb{R}^d$, and the error dynamics become:

$$e_{t+1} = (I - 2\eta x_i x_i^\top) e_t, \tag{33}$$

where x_i and e_{t+1} are *random variables* condition on e_t due to stochastic sampling. Remember that $e_t = w_t - w^*$, and the error term is just a change-of-variables of the raw parameters that is easier to analyze.

To show that our loss will converge to zero, we'll first show that loss is bounded below by zero. Then, we'll show that each iteration will always reduce the global loss (in expectation). With both conditions satisfied, the global loss must converge to zero.

The first condition is simple, and is true by the nature of the quadratic loss. In terms of our error term, the loss is:

$$L(e_t) = \|X e_t\|^2 = e_t^\top X^\top X e_t \geq 0. \tag{34}$$

The second condition will require more machinery. We want to show that $L(e_{t+1}) < L(e_t)$ in expectation. To do this, let's decompose a single step of the loss, noting that the improvement contains a linear term A and a quadratic term B:

$$\begin{aligned} L(e_{t+1}) &= L(e_t + e_{t+1} - e_t) && (35) \\ &= L(e_t) + \underbrace{2e_t^\top X^\top X(e_{t+1} - e_t)}_{\text{Linear term } A} + \underbrace{(e_{t+1} - e_t)^\top X^\top X(e_{t+1} - e_t)}_{\text{Quadratic term } B}. && (36) \\ & && (37) \end{aligned}$$

We will now show that, under specific conditions, $E[A + B | e_t] < 0$, guaranteeing an expected improvement

 All Rights Reserved. This may not be publicly shared without explicit permission.

at every step. First, let's establish a bound on the linear term A:

$$E[A|e_t] = 2e_t^\top X^\top X E[e_{t+1} - e_t|e_t] \tag{38}$$
$$= 2e_t^\top X^\top X(-2\eta)E[x_i x_i^\top]e_t \tag{39}$$
$$= 2e_t^\top X^\top X(-2\eta)(1/n)(X^\top X)e_t \tag{40}$$
$$= -\frac{4\eta}{n} e_t^\top (X^\top X)^2 e_t \tag{41}$$
$$\leq -\frac{4\eta}{n}\lambda_{\min}^2 L(e_t), \tag{42}$$

where $\lambda_{\min}$ is the smallest eigenvalue of $X^\top X$.

Next, let's establish a bound on the quadratic term B:

$$E[B|e_t] = E[(e_{t+1} - e_t)^\top X^\top X(e_{t+1} - e_t)|e_t] \tag{43}$$
$$= E[(-2\eta x_i x_i^\top e_t)^\top X^\top X(-2\eta x_i x_i^\top e_t)|e_t] \tag{44}$$
$$\leq 4\eta^2\lambda_{\max}^2 e_t^\top E[x_i x_i^\top x_i x_i^\top]e_t \tag{45}$$
$$\leq 4\eta^2\lambda_{\max}^2 f^2 e_t^\top E[x_i x_i^\top]e_t \tag{46}$$
$$\leq 4\eta^2\lambda_{\max}^2 f^2 e_t^\top (1/n)(X^\top X)e_t \tag{47}$$
$$\leq 4\eta^2\lambda_{\max}^3 f^2 L(e_t), \tag{48}$$

where $\lambda_{\max}$ is the largest eigenvalue of $X^\top X$, and $f = \max_i \|x_i\|^2$ is the maximum squared norm of any data point. Notably, $\lambda_{\min}$, $\lambda_{\max}$, and f are all constants from the dataset, and do not change during training.

Combining both bounds, our expected loss at each step is:

$$E[L(e_{t+1})|e_t] \leq (1 - \underbrace{\eta\frac{4}{n}\lambda_{\min}^2}_{\text{From } A} + \underbrace{\eta^2 4\lambda_{\max}^3 f^2}_{\text{From } B}) L(e_t), \tag{49}$$

which, for a small enough learning rate η, is always less than $L(e_t)$. Thus, we can claim converge of SGD.

8 Momentum

Another way to tackle the issue of ill-conditioning is to use **momentum**. The idea of momentum is to accumulate a running average of the gradients, and use this average to update the parameters. This has the effect of smoothing out the updates, and allowing for larger learning rates. The update rule for gradient descent with momentum is given by:

$$m_{t+1} = \beta m_t + (1 - \beta)\nabla_\theta \mathcal{L}(\theta_t) \tag{50}$$
$$\theta_{t+1} = \theta_t - \eta m_{t+1}, \tag{51}$$

where m_t is a per-parameter momentum term, and $\beta \in (0, 1)$.

For our specific case of least-squares linear regression, momentum results in an almost *square-root* reduction of the condition number, and enables the usage of a 2x higher learning rate. As this derivation is somewhat involved, we refer to this excellent blog post for details: https://distill.pub/2017/momentum/.

 All Rights Reserved. This may not be publicly shared without explicit permission.

9 The Adam Optimizer

We know that Gradient Descent (GD) has a property that it takes larger steps in directions with larger singular values and smaller steps in directions with small singular values. This has advantages and disadvantages.

- **Pro:** Combined with early stopping, this leads to *implicit regularization*.
- **Con:** It makes choosing a single learning rate η difficult — specifically, the learning rate is bottlenecked by the largest singular value, and therefore directions with 'medium' singular values may show slow progress. (It is ok for us if small singular values show slow progress, this is the implicit regularization feature.)

One way to solve this problem would be to normalize the step in each direction by the size of the singular value. While this removes the "implicit regularization" it may have other benefits. But there is a key challenge, computing the SVD is expensive.

9.1 Normalizing Gradients: SignGD

A more practical approach is to normalize the gradients in the native coordinate basis, instead of in the SVD basis. Instead of taking a step proportional to the gradient $\nabla L(\mathbf{w})$, we can take a step in the direction of the **sign** of the gradient. Thus, steps in all directions of the coordinate basis are the same size.

This can be implemented as:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\nabla L(\mathbf{w}_t)}{\sqrt{(\nabla L(\mathbf{w}_t))^2 + \epsilon}}$$

where the square is applied element-wise, and ϵ is a small constant for numerical stability. The term $\frac{\mathbf{m}}{\sqrt{\mathbf{v}+\epsilon}}$ effectively computes the sign of each component of the gradient vector. This is called **SignGD**.

One problem with SignGD is that near an optimum, the sign of the gradient can flip back and forth, causing the algorithm to oscillate and never fully converge. However, SignGD is a useful building block to help us think about the behavior of Adam, one of the widely used algorithms. This trick of using the sign of the gradient instead can also be used with Stochastic Gradient Descent, which gives us SignSGD.

9.2 Adam: Adaptive Moment Estimation

Adam improves upon SignSGD by smoothing out the updates using ideas from momentum. It maintains exponential moving averages of two quantities:

(a) The gradient itself (the first moment estimate), $\mathbf{m}_t$.

(b) The element-wise square of the gradient (the second moment estimate), $\mathbf{v}_t$.

The updates are:

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla L(\mathbf{w}_t)$$
$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla L(\mathbf{w}_t))^2$$

where β_1 and β_2 are hyperparameters.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Bias Correction: Since $\mathbf{m}_0$ and $\mathbf{v}_0$ are initialized to zero, the estimates are biased towards zero, especially at the beginning of training. To correct for this bias we define:

$$\tilde{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1-\beta_1^{t+1}} \qquad \tilde{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1-\beta_2^{t+1}}$$

As t increases, the denominators approach 1, and the correction vanishes.

This gives us the final update rule for Adam:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\tilde{\mathbf{m}}_{t+1}}{\sqrt{\tilde{\mathbf{v}}_{t+1}} + \epsilon}$$

Memory Cost: Adam requires more memory than simpler optimizers.

- SGD: Stores weights (1x).
- SGD with Momentum: Stores weights and momentum vector (2x).
- Adam: Stores weights, momentum vector, and squared gradient vector (3x).

9.3 AdamW: Adam with Decoupled Weight Decay

Weight decay (L2 regularization) is a common technique to prevent overfitting. In standard GD, this is equivalent to adding a penalty $\frac{\lambda}{2}\|\mathbf{w}\|_2^2$ to the loss. The gradient update for Ridge Regression is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta(\boldsymbol{\nabla} L_{data}(\mathbf{w}_t) + \lambda \mathbf{w}_t) = (1-\eta\lambda)\mathbf{w}_t - \eta \boldsymbol{\nabla} L_{data}(\mathbf{w}_t)$$

If we simply add the $\lambda \mathbf{w}_t$ term to the gradient in Adam, it gets included in the adaptive normalization ($\mathbf{v}_t$). This couples the effect of the learning rate η and the regularization strength λ in an undesirable way.

AdamW decouples the weight decay from the adaptive gradient update. The weight decay is applied directly to the weights, independently of the gradient moments.

$$\mathbf{w}_{t+1} = (1-\lambda')\mathbf{w}_t - \eta \frac{\tilde{\mathbf{m}}_{t+1}}{\sqrt{\tilde{\mathbf{v}}_{t+1}} + \epsilon}$$

Here, λ' is a separate weight decay hyperparameter. This has been shown to improve generalization performance.

10 Gradient Descent as Penalized Linear Improvement

Up to this point, we have viewed gradient descent as an iterative algorithm. However, it is also useful to view gradient descent as the solution to a sequence of **local optimization problems**.

To see this, it is useful to consider the Taylor approximation of the function we are trying to minimize (in the case of neural networks, this is the loss function). We can write the Taylor expansion of the loss function around some θ as:

$$\mathcal{L}(\theta + \Delta\theta) = \mathcal{L}(\theta) + \left(\nabla_\theta \mathcal{L}(\theta)\right)^\top \Delta\theta + \text{higher order terms.} \tag{52}$$

Gradient descent takes a step to minimize the loss based on this linear approximation. The direction of the negative gradient tells us the direction of the step. The stepsize η is chosen by us. We have seen how the choice of η is connected to the singular values of the data matrix. There are many ways in which η is

 All Rights Reserved. This may not be publicly shared without explicit permission.

chosen (e.g. using the smoothness and/or strong convexity of the function, a learning rate schedule for SGD, line search etc.) and we won't go into them here. We focus on a particular perspective following **?**. The essentially propose a 'recipe' for optimizers that we illustrate below.

We note that there are two competing objectives in choosing the step we take with gradient descent.

- We want to make sure our steps are small enough that the local linear approximation continues to hold.
- We want to make sure that we take large steps so that we converge to the minimum quickly.

Why not formulate this then as an optimization problem? Let $||\cdot||$ denote a general norm and we will specify our choice shortly. Then our constrained optimization problem is to minimize the value of the function while making sure that

$$u = \underset{||\Delta\theta||<\eta}{\operatorname{argmin}} \ \mathcal{L}(\theta) + \big(\nabla_\theta \mathcal{L}(\theta)\big)^\top \Delta\theta. \tag{53}$$

Note that $\mathcal{L}(\theta)$ is a constant in the expression above, and this is equivalent to:

$$u = \underset{||\Delta\theta||<\eta}{\operatorname{argmin}} \ \big(\nabla_\theta \mathcal{L}(\theta)\big)^\top \Delta\theta. \tag{54}$$

In this framework, we can now explore other choices for the norm. Different norms have different geometric properties and hence some may be more suited to particular problems.

10.1 Example 1: Euclidean Norm

We start with the Euclidean norm.

- **Geometry:** The constraint confines the update to a hypersphere of radius η.

$$u = \underset{||\Delta\theta||_2<\eta}{\operatorname{argmin}} \ \mathcal{L}(\theta) + \big(\nabla_\theta \mathcal{L}(\theta)\big)^\top \Delta\theta. \tag{55}$$

The Cauchy-Schwartz Inequality gives us a closed form solution for this,

$$\Delta\theta = \eta \frac{\nabla\mathcal{L}(\theta)}{||\nabla\mathcal{L}(\theta)||_2}. \tag{56}$$

We observe this to be a scaled version of gradient descent — where the step is in the direction of the gradient, but is normalized by the gradient magnitude. This has some nice properties, namely that the stepsize η is the only parameter controlling the size of the step. On the other hand, even as the gradient gets close to zero, the size of the step does not decay and this might lead to bouncing around near the minimum.

We could consider the Lagrange Multipliers/Regularization perspective and formulate our optimization problem to be:

$$v = \underset{\Delta\theta}{\operatorname{argmin}} \ \big(\nabla_\theta \mathcal{L}(\theta)\big)^\top \Delta\theta + \frac{1}{2\eta}\|\Delta\theta\|^2. \tag{57}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

Note: the formulation above and the constrained optimization formulation in 53 sweep out the same family of solutions (subject to scaling/choice of η) even though the problems are not equivalent.

Since this is a convex quadratic, we can find the closed form solution by taking the gradient and setting it to zero.

$$\nabla_{\Delta\theta}\left(\left(\nabla_\theta \mathcal{L}(\theta)\right)^\top \Delta\theta + \frac{1}{2\eta}\|\Delta\theta\|^2\right) = \left(\nabla_\theta \mathcal{L}(\theta)\right) + \frac{1}{\eta}\Delta\theta = 0 \tag{58}$$

$$\Delta\theta = -\eta\left(\nabla_\theta \mathcal{L}(\theta)\right). \tag{59}$$

This is exactly the standard gradient descent rule!

Thus, we can think of gradient descent as finding the best improvement to the function, subject to a Euclidean norm constraint. It is comforting to recover something familiar in this case — this likely means that the approach itself may have merit!

10.2 Example 2: Infinity Norm

For convenience we use the notation $g = \nabla_\theta \mathcal{L}(\theta)$ for the gradient of the loss. This gives the optimization problem:

$$u = \underset{||\Delta\theta||_\infty \leq \eta}{\operatorname{argmin}} \; g^T \Delta\theta.$$

We consider a bound on the **infinity norm** of the update, i.e., $||\Delta\theta||_\infty = \max_i |\Delta\theta_i|$. Thus the infinity norm is bounding the max value of the entires of the vector $\Delta\theta_i$.

We note:

- **Geometry:** This constraint confines the update $\Delta\boldsymbol{\theta}$ to a hypercube centered at the origin, with side length 2η.
- **Optimization Problem:** The objective is linear, and the constraints ($-\eta \leq \Delta\boldsymbol{\theta}[j] \leq \eta$ for all j) are also linear. This is a **Linear Program**. The solution must lie at one of the vertices (corners) of the hypercube.

This results in the following update rule:

$$u = \underset{||\Delta\theta||_\infty \leq \eta}{\operatorname{argmin}} \; g^T \Delta\theta = -\eta \cdot \operatorname{sign}(g). \tag{60}$$

To see this note that $||\Delta\theta||_\infty \leq \eta$ means that $\Delta\theta \in [-\eta, \eta]^d$. We are interested in minimizing $g^\top \Delta\theta = \sum_{i=1}^d g_i \Delta\theta_i$. We know that since this is a convex polytope the optimum will be at a corner. To maximize each term in this sum, we can match the sign of $\Delta\theta_i$ with the sign of g_i, and any sign mismatch between $\Delta\theta_i$ and g_i, will only decrease the sum. This can be formalized using a dual norm argument, as we do in 127, but we skip here for now. This recovers the **Sign Gradient Descent (SignGD)** algorithm we discussed earlier!

Now, recall our motivation for Adam. We had observed the implicit regularization of gradient descent and that Gradient descent takes larger steps in directions corresponding to larger singular values. This had the advantage of not moving too much in directions that the data does not represent, i.e. those directions corresponding to smaller singular values. However, this also has some disadvantages. Specifically, it can

 All Rights Reserved. This may not be publicly shared without explicit permission.

cause slow convergence in the directions corresponding to singular values that are... *medium* sized ... not very large but also not very tiny. These are not directions we want to entirely ignore. If you recall, having more uniform steps along different singular vectors was one of the key properties Adam, which you can think of as a variation on SignGD.

10.3 Example 3: The Spectral Norm and the Shampoo Optimizer

The perspective so far treats all parameters $\boldsymbol{\theta}$ as a single, undifferentiated vector. However, neural networks have a distinct structure: they are composed of layers, each with its own weight matrix W and bias vector $\mathbf{b}$. Ideas such as Xavier initialization differentiate the W and the b. Can optimizers benefit from taking a look at the underlying structure of the $\boldsymbol{\theta}$ parameters? The scaling issues that motivate methods like Xavier initialization primarily arise from the multiplication by weight matrices.

We focus on the matrices W and think of what a gradient steps means for them. Then we are looking for a step ΔW that is appropriately small.

The optimization problem becomes:

$$\Delta W^* = \underset{||\Delta W|| \leq \eta}{\arg\min} \langle \nabla_W L(W), \Delta W \rangle_F \tag{61}$$

Here, the inner product is the Frobenius inner product for matrices, defined as $\langle A, B \rangle_F = \mathrm{Tr}\Big(A^T B\Big)$.

We consider using the **spectral norm** (also known as the operator 2-norm) to bound ΔW, which measures the maximum scaling a matrix can apply to a unit vector.

$$||A||_2 = \max_{\|\mathbf{x}\|_2 = 1} \|A\mathbf{x}\|_2 = \sigma_{\max}(A) \quad \text{(the largest singular value of } A\text{)} \tag{62}$$

Our optimization problem is now:

$$\Delta W^* = \underset{||\Delta W||_2 \leq \eta}{\arg\min} \langle \nabla_W L(W), \Delta W \rangle_F \tag{63}$$

Let $G = \nabla_W L(W)$. Now we want to solve

$$\underset{||B||_2 \leq 1}{\arg\min} \mathrm{Tr}\Big(G^T B\Big)$$

to find the optimal update.

Let the SVD of G be $G = U\Sigma V^T$.

$$\begin{aligned}
\mathrm{Tr}\Big(G^T B\Big) &= \mathrm{Tr}\Big(V\Sigma^T U^T B\Big) \\
&= \mathrm{Tr}\left(\left(\sum_i \sigma_i \mathbf{v}_i \mathbf{u}_i^T\right) B\right) && \text{(Outer product form of SVD for } G^T\text{)} \\
&= \sum_i \sigma_i \,\mathrm{Tr}\Big(\mathbf{v}_i \mathbf{u}_i^T B\Big) && \text{(Linearity of trace)} \\
&= \sum_i \sigma_i \,\mathrm{Tr}\Big(\mathbf{u}_i^T B \mathbf{v}_i\Big) && \text{(Cyclic property of trace)}
\end{aligned}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

$$= \sum_i \sigma_i(\mathbf{u}_i^T B \mathbf{v}_i) \qquad \text{(Trace of a scalar is the scalar itself)}$$

By the Cauchy-Schwarz inequality and the definition of the spectral norm:

$$|\mathbf{u}_i^T(B\mathbf{v}_i)| \leq \left\|\mathbf{u}_i^T\right\|_2 \|B\mathbf{v}_i\|_2 \leq 1 \cdot (\|B\|_2 \|\mathbf{v}_i\|_2) \leq 1 \cdot (1 \cdot 1) = 1$$

. Thus, each term $\mathbf{u}_i^T B \mathbf{v}_i \leq 1$. This gives an upper bound on the trace:

$$\mathrm{Tr}\Big(G^T B\Big) \leq \sum_i \sigma_i.$$

This bound is attained when we choose $B = UV^T$. In this case, $\mathbf{u}_i^T B \mathbf{v}_j = \mathbf{u}_i^T U V^T \mathbf{v}_j = \mathbf{e}_i^T \mathbf{e}_j = \delta_{ij}$. The trace becomes $\sum_i \sigma_i(1) = \sum_i \sigma_i$. Therefore, the maximizer of the inner product is is UV^T, i.e.

$$UV^T = \underset{||B||_2 \leq 1}{\arg\max}\, \mathrm{Tr}\Big(G^T B\Big)$$

.

But we were interested in the minimizer, which we can obtain by replacing B with $-B$. Thus

$$\mathrm{Tr}\Big(G^T B\Big) \geq -\sum_i \sigma_i, and$$

$$-UV^T = \underset{||B||_2 \leq 1}{\arg\min}\, \mathrm{Tr}\Big(G^T B\Big).$$

Note: One can think of UV^T are the orthogonalization of the matrix B — we keep only the directions and lose all the magnitudes — this is a matrix-parallel to SignGD.

Note: The issue with the naive orthogonalization procedure is that performing an SVD is expensive, and we don't want to have to perform this at every iteration. A key innovation of the Shampoo optimizer was to approximate UV^T efficiently. The update in the **Shampoo** optimizer can be derived by noting the orthogonalization of ∇_W can be also be constructed by taking the inverse fourth-root of the left and right "covariance" matrices:

$$(GG^T)^{-1/4}\, G\, (G^T G)^{-1/4} = (U\Sigma^2 U^T)^{-1/4}\, U\Sigma V^T\, (V\Sigma^2 V^T)^{-1/4} = UV^T. \tag{64}$$

The approximation in Shampoo is that the left and right factors can be cached and used across multiple gradient steps, preventing the need for an expensive matrix decomposition at each step. In the full Shampoo algorithm, it is typical to utilize an exponential moving average of the GG^T and $G^T G$ matrices.

11 Towards a Principled Choice of Norm for Deep Learning

Our recipe can generate optimizers, but how do we choose the "right" norm for deep neural networks? Inspiration can be found by revisiting initialization.

 All Rights Reserved. This may not be publicly shared without explicit permission.

11.1 Inspiration from Xavier Initialization

The goal of Xavier initialization was to ensure that activation do not blow up or shrink down as they pass through a layer. If inputs $x_i \sim \mathcal{N}(0,1)$, we want the outputs to also have a similar distribution. This requires scaling the initial weights by the fan-in — if more things are being added together then we want to make sure they are all small enough so that our output does not blow up. This suggests that a good norm should be one that naturally accounts for the dimensions of the network layers.

Standard norms do not capture this. For example, preserving the L_2 norm ($\|\mathbf{x}\|_2 = 1 \implies \|W\mathbf{x}\|_2 \approx 1$) is not what Xavier achieves, because the input and output dimensions are different.

11.2 The Root Mean Square (RMS) Norm

A norm that aligns with the goal of Xavier is the **Root Mean Square (RMS) norm**:

$$\|\mathbf{x}\|_{RMS} = \frac{1}{\sqrt{d}}\|\mathbf{x}\|_2 = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2} \tag{65}$$

If$\|\mathbf{x}\|_{RMS} = 1$, the entries of $\mathbf{x}$ have an average squared magnitude of 1. This captures the idea of preserving the size of the entries of the vector.

11.3 The Induced RMS-to-RMS Matrix Norm

We can now define an induced matrix norm based on the RMS norm, which measures how a matrix scales vectors from an RMS-normed input space to an RMS-normed output space.

$$\|A\|_{RMS\to RMS} = \max_{\|\mathbf{x}\|_{RMS}=1} \|A\mathbf{x}\|_{RMS}\,. \tag{66}$$

Let A be a $d_2 \times d_1$ matrix (d_1 = fan-in, d_2 = fan-out). Turns out, the RMS norm is related to a norm we know and understand well, the spectral norm. First we note that the constraint$\|\mathbf{x}\|_{RMS} = 1$ is equivalent to $\|\mathbf{x}\|_2 = \sqrt{d_1}$. Then consider:

$$\begin{aligned}
\|A\|_{RMS\to RMS} &= \max_{\|\mathbf{x}\|_2=\sqrt{d_1}} \frac{1}{\sqrt{d_2}}\|A\mathbf{x}\|_2 \\
&= \frac{1}{\sqrt{d_2}} \max_{\|\mathbf{z}\|_2=1} \left\|A(\sqrt{d_1}\mathbf{z})\right\|_2 \quad \text{(substituting } \mathbf{x} = \sqrt{d_1}\mathbf{z}\text{)} \\
&= \frac{\sqrt{d_1}}{\sqrt{d_2}} \max_{\|\mathbf{z}\|_2=1} \|A\mathbf{z}\|_2 \\
&= \sqrt{\frac{d_1}{d_2}}\|A\|_2 = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\|A\|_2
\end{aligned}$$

This new norm is just a scaled version of the spectral norm. The scaling factor depends directly on the architecture of the layer (its fan-in and fan-out).

 All Rights Reserved. This may not be publicly shared without explicit permission.

12 Maximal Update Parameterization

With our newfound acceptance of the concept of *picking an appropriate norm*, we can now consider that norms over parameters may not be the best choice. Rather, we may want to consider norms over the **activations** of our network. One motivation for this choice is that nonlinearities such as sigmoid or tanh are sensitive to the scale of their inputs, and we would like to regularize this. Another motivation is that activations are in some sense the "communication" between subsequent layers, and we would like to ensure that this communication is well-behaved.

To measure the size of an activation vector, we will rely on the **root mean-squared (RMS)** norm, which is defined as:

$$||x||_{\text{RMS}} = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}. \tag{67}$$

The RMS norm is similar to the Euclidean norm, but is normalized by the number of elements in the vector. This enables the RMS norm to scale with $O(1)$ in the size of individual elements. A d-length vector composed entirely of ones will have an RMS norm of 1, whereas its Euclidean norm will scale with $\sqrt{d}$. For a zero-mean vector, the RMS norm can be equated to the standard deviation of the elements (and the squared RMS norm is the variance).

By considering the RMS norm of activations through the network, we can establish the "proper" scaling factors for weights, learning rates, and other hyperparameters. For this section, we will focus on the big-O scaling with respect to *layer width* d, and ignore other factors.

The core problem that **Maximal Update Parameterization (μP)** aims to solve is this:

> Can we find an efficient way to select hyperparameters for a very large model without performing full training runs on it?

The idea behind μP is to develop a method for *scaling* or *transferring* hyperparameters found on a small, cheap-to-train "proxy" model to a much larger target model in a principled way. This lecture builds the intuition for how such a scaling can be derived.

Xavier: Let's consider a feedforward neural network with N dense layers of $W \in \mathbb{R}^{d\times d}$, and ReLU activations. Assume the input is a vector of all ones. If W is initialized elementwise from a unit Gaussian, what happens to the RMS norm of the activations as we propagate through the network? Since RMS norms scale in proportion to the norms of individual elements, we can analyze the expected norm of a single output element in the activation vector. A single output element is defined as a dot product product between a row of W and the input vector, which is a sum of d independent Gaussian random variables – i.e., a random variable with a standard deviation of $\sqrt{d}$. This means that at each layer, the RMS norm of the activations will increase by a factor of $\sqrt{d}$, and will blow up over time. This is known as the **exploding gradient** problem.

Instead, if we scale down each W matrix by $1/\sqrt{d}$, then the standard deviation of each output element will be 1, and the RMS norm of the activations will remain stable throughout the network. This is known as the **Xavier initialization** scheme, and is a common default choice for initializing weights in a neural network.

To understand how mu-P works let us go back to our general recipe (matrix-version) for an optimizer from Bernstein-Newhouse:

$$\Delta W^* = \underset{\|\Delta W\|\leq\eta}{\arg\min}\langle \nabla_W L(W), \Delta W\rangle$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

In this recipe, we have two choices. Our first choice is to choose the appropriate norm $\|\cdot\|$, and the second is to choose the constraint η. We have explored different choices of the norm, but how does one choose η? This likely requires a hyperparameter sweep, which is something we would like to avoid. However, the RMS norm has a very nice property — a built in dependence on the dimensions of the associated matrix.

Thus, if we constrain:

$$\|\Delta W\|_{RMS\to RMS} \leq \gamma$$

This is equivalent to constraining the spectral norm:

$$\sqrt{\frac{d_{in}}{d_{out}}}\|\Delta W\|_2 \leq \gamma \implies \|\Delta W\|_2 \leq \gamma\sqrt{\frac{d_{out}}{d_{in}}},$$

in a way that depends on d_{out} and d_{in}.

What happens if we use this RMS-to-RMS norm in our optimizer recipe? The constraint on the update matrix for each layer becomes:

$$\begin{aligned}\|\Delta W\|_{RMS\to RMS} &\leq \gamma \\ \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\|\Delta W\|_2 &\leq \gamma \\ \implies \|\Delta W\|_2 &\leq \gamma\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\end{aligned}$$

This is key. By choosing a single, global hyperparameter γ, we induce a layer-specific constraint on the update size. The effective learning rate for each layer is automatically scaled by its geometry (its d_{in} and d_{out}). This gives us layer-specific behavior from a single global hyperparameter and addresses the problem that different layers in a network may require updates of very different magnitudes.

This principle is the essential idea behind **Maximal Update Parametrization (μP)** and related optimizers like **Muon**, which aim to make hyperparameter tuning, especially the learning rate, more stable and transferable across different model sizes.

We can build on this intuition to get to the Muon optimizer, but first, let us complete the story on (μP). Specifically, what happens when we care about layer-specific learning rate scaling, but want to use an optimizer such as Adam or SGD?

13 Deriving Learning Rate Scaling for Adam/SignSGD

Instead of deriving a completely new optimizer, we can use the insights from the RMS norm to improve our choice of step size for existing optimizers like Adam or SignSGD. For simplicity, we'll analyze **SignSGD**:

$$W_{t+1} = W_t - \eta \cdot \text{sign}(\nabla_W L(W)).$$

Our goal is to choose the learning rate η intelligently, factoring in layer architectures. We'll do this by enforcing the constraint that the update step

$$\Delta W = -\eta \cdot \text{sign}(\nabla_W L(W)),$$

has a bounded RMS-to-RMS norm:

$$\|\Delta W\|_{RMS\to RMS} \leq \gamma.$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

We consider an update with a mini-batch size of 1 for ease of analysis.

Observation 1: The Gradient is Rank-1. For a single data point and common loss functions (like squared error or cross-entropy), the gradient of the loss with respect to the weights, $\nabla_W L$, is a **rank-1 matrix**. This is because the update can only be in the span of that single data point.

Now any rank-1 matrix can be written as an outer product:

$$\nabla_W L = \sigma \mathbf{u}\mathbf{v}^T.$$

Furthermore, note that for a rank-1 matrix, the spectral norm and Frobenius norm are equal: $\|\nabla_W L\|_2 = \|\nabla_W L\|_F = \sigma$.

Observation 2: The Sign Matrix is Rank-1. Let $S = \text{sign}(\nabla_W L)$ be the element-wise sign matrix. Since the sign of a product is the product of signs, we have:

$$S = \text{sign}(\sigma \mathbf{u}\mathbf{v}^T) = \text{sign}(\mathbf{u}\mathbf{v}^T) = (\text{sign}(\mathbf{u}))(\text{sign}(\mathbf{v}^T))$$

This shows that S is also a rank-1 matrix, as it is the outer product of two vectors.

Deriving the Scaling Law. Now we combine these observations to find the appropriate scaling for η.

(a) We want $\|\Delta W\|_{RMS\to RMS} \leq \gamma$. This is the same as

$$\|\eta S\|_{RMS\to RMS} \leq \gamma \implies \eta \cdot \|S\|_{RMS\to RMS} \leq \gamma$$

(b) We can replace with the spectral norm:

$$\eta \cdot \sqrt{\frac{d_{in}}{d_{out}}} \|S\|_2 \leq \gamma$$

(c) Since S is rank-1, its spectral norm is equal to its Frobenius norm. The matrix $S \in \mathbb{R}^{d_{out}\times d_{in}}$ contains only entries of $+1$ or -1, and this is easy to calculate.

$$\|S\|_2^2 = \|S\|_F^2 = \sum_{i=1}^{d_{out}} \sum_{j=1}^{d_{in}} S_{ij}^2 = \sum_{i=1}^{d_{out}} \sum_{j=1}^{d_{in}} 1^2 = d_{in} \cdot d_{out}$$

Therefore, $\|S\|_2 = \sqrt{d_{in} d_{out}}$.

(d) Substitute this back in gives a bound for η:

$$\eta \cdot \sqrt{\frac{d_{in}}{d_{out}}} \cdot \sqrt{d_{in} d_{out}} \leq \gamma$$
$$\eta \leq \frac{1}{d_{in}} \cdot \gamma$$

This captures a core insight of μP, matching what you see in Table 3 (under Adam) of Yang et. al's *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*. While this paper uses

 All Rights Reserved. This may not be publicly shared without explicit permission.

far more complicated mathematics to justify the scaling, we build on the perspective of Bernstein, Simon and Yang. The analysis above is not in this paper but is a new derivation by the instructors as one that illustrates the essential point.

Note on Larger Batch Sizes: If the batch size is greater than 1, the gradient matrix may no longer be rank-1. In this case, $\|S\|_2 \leq \|S\|_F$, so using the Frobenius norm in our derivation yields an upper bound. The resulting scaling law still holds as a conservative bound, meaning it would suggest a slightly smaller (safer) learning rate.

13.1 What is Parameterization and Why Does It Matter?

So why do we care about parameterization? Parameterization is about choosing the right "units" or "scaling" to analyze a system. The goal of μP is to find a parameterization of a neural network such that its optimal hyperparameters (like learning rate) remain stable as the network's width changes, so that we can optimize hyperparameters by training smaller neural networks and then 'transfer' these hyperparameters to larger networks.

We illustrate the point on choosing the 'units' by building on an example from Yang et al. Consider summing n i.i.d. random variables $X_1, \ldots, X_n$ with zero mean and unit variance. We want to understand how this behaves 'on average'. How we choose to scale matters, specifically:

- $\frac{1}{n}\sum_{i=1}^{n} X_i \to 0$ — this goes to zero.
- $\frac{1}{1}\sum_{i=1}^{n} X_i \to \infty$ — this diverges.
- But with the **right scaling** $\frac{1}{\sqrt{n}}\sum_{i=1}^{n} X_i \xrightarrow{d} \mathcal{N}(0,1)$. This scaling reveals a non-trivial, stable behavior in the limit.

Just as $1/\sqrt{n}$ is the "right" scaling to understand the sum of random variables, μP provides the "right" scaling (e.g., $\eta \propto 1/d_{in}$) to understand how hyperparameters behave as network width changes.

14 Bernstein, Simon, Yang

For a separate perspective behind the μ-P scaling laws, we can adopt a simplified linear algebraic perspective following "A Spectral Condition for Feature Learning" by Bernstein, Simon, and Yang. This perspective arrives at the same scaling laws as the original MUP paper, which used more complex tensor algebra.

Let $\mathbf{h}_l$ be the activation vector of hidden layer l.

The output activation h_l is computed from the previous layer's activation h_{l-1}:

$$h_l = \sigma(W_l h_{l-1} + b_l)$$

where $W_l \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix, and $h_{l-1} \in \mathbb{R}^{d_{in}}$. Our focus is on how parameters should scale with the network width (d_{in}, d_{out}), not depth.

We can formalize the goal of stable training dynamics with two desiderata.

(a) **Stable Feature Scale:** The magnitude of activations should not explode or vanish as the network

 All Rights Reserved. This may not be publicly shared without explicit permission.

grows. We want the RMS norm to be of constant order with respect to width:

$$\|\mathbf{h}_l\|_{RMS} = \Theta(1).$$

This is the core idea behind Xavier initialization.

(b) **Meaningful Updates:** The layers should continue to learn effectively as the network grows. This means the change in activations per step, $\Delta\mathbf{h}_l$, should also be of a constant, non-negligible order:

$$\|\Delta\mathbf{h}_l\|_{RMS} = \Theta(1).$$

14.0.1 Deriving Scaling Conditions

Let us see what it takes to achieve these properties, in a simplified linear setting. Assume $h_l = W_l h_{l-1}$. We proceed with an inductive argument. Assume the desired properties hold for layer $(l-1)$. What conditions on W_l are necessary for them to hold at layer l?

Condition on Activations: The scaling of a vector by a matrix is governed by the matrix's singular values. It can be shown that during training, hidden activations tend to align with the direction of the largest singular vectors of the weight matrices (out of scope here, please see the paper). Assuming this alignment, the relationship between the norms becomes:

$$\|h_l\|_2 \approx \|W_l\|_2 \|h_{l-1}\|_2$$

where $\|W_l\|_2$ is the spectral norm (largest singular value) of W_l.

Using our desired property for h_l and the inductive assumption for h_{l-1}:

$$\|h_l\|_{RMS} = \frac{1}{\sqrt{d_{out}}}\|h_l\|_2 \approx \mathcal{O}(1) \implies \|h_l\|_2 \approx \mathcal{O}(\sqrt{d_{out}})$$

$$\|h_{l-1}\|_{RMS} \approx \mathcal{O}(1) \implies \|h_{l-1}\|_2 \approx \mathcal{O}(\sqrt{d_{in}})$$

Substituting these into the norm relationship gives a condition on the spectral norm of W_l:

$$\mathcal{O}(\sqrt{d_{out}}) \approx \|W_l\|_2 \cdot \mathcal{O}(\sqrt{d_{in}}) \implies \|W_l\|_2 \approx \mathcal{O}\left(\sqrt{\frac{d_{out}}{d_{in}}}\right)$$

This is equivalent to stating that the RMS-to-RMS induced norm $||W_l||_{RMS\to RMS}$ should be $\mathcal{O}(1)$.

Condition on Updates: Now consider the update to the hidden layer, Δh_l. Applying the product rule to $h_l = W_l h_{l-1}$:

$$\Delta h_l = (\Delta W_l)h_{l-1} + W_l(\Delta h_{l-1})$$

Taking the norm and applying the triangle inequality:

$$\|\Delta h_l\|_2 \leq \|\Delta W_l h_{l-1}\|_2 + \|W_l \Delta h_{l-1}\|_2 \approx \|\Delta W_l\|_2 \|h_{l-1}\|_2 + \|W_l\|_2 \|\Delta h_{l-1}\|_2$$

For $\|\Delta h_l\|_{RMS}$ to be $\mathcal{O}(1)$, we need $\|\Delta h_l\|_2$ to be $\mathcal{O}(\sqrt{d_{out}})$. Using our inductive assumptions and derived conditions:

 All Rights Reserved. This may not be publicly shared without explicit permission.

- $\|h_{l-1}\|_2 \approx \mathcal{O}(\sqrt{d_{in}})$
- $\|W_l\|_2 \approx \mathcal{O}(\sqrt{d_{out}/d_{in}})$
- $\|\Delta h_{l-1}\|_2 \approx \mathcal{O}(\sqrt{d_{in}})$ (from the inductive assumption on the update RMS norm)

For the entire expression to be $\mathcal{O}(\sqrt{d_{out}})$, the term $\|\Delta W_l\|_2\|h_{l-1}\|_2$ must also be of this order. This implies a condition on the update to the weight matrix:

$$\|\Delta W_l\|_2 \cdot \mathcal{O}(\sqrt{d_{in}}) \approx \mathcal{O}(\sqrt{d_{out}}) \implies \|\Delta W_l\|_2 \approx \mathcal{O}\left(\sqrt{\frac{d_{out}}{d_{in}}}\right)$$

This means the RMS-to-RMS norm of the weight update ΔW_l should also be $\mathcal{O}(1)$.

Under these conditions, we can achieve meaningful 'feature learning.' These derived conditions on the weights and their updates form the basis for μ-P's scaling laws for hyperparameters like the learning rate. The paper goes on to do an analysis of Gradient Descent (Not SignGD/Adam) and shows that $\eta_\ell = \sqrt{\frac{d_{out}}{d_{in}}}$ is the right scaling, but we did not have time to discuss this in class and is out of scope.

15 The Muon Optimizer

Muon (Momentum Orthogonalized by Newton Schulz) is a modern optimizer that has demonstrated significant performance improvements, notably in the "nanoGPT speedrun" competition. It represents a shift towards architecture-specific optimizers that are aware of the network's dimensions.

15.1 Key Idea 1: The RMS-to-RMS Norm

The first key idea of Muon is to replace the spectral norm in the optimizer recipe with the **RMS-to-RMS induced norm**. This norm is simply a scaled version of the spectral norm:

$$\|\Delta W\|_{RMS\to RMS} = \sqrt{\frac{d_{in}}{d_{out}}}\|\Delta W\|_2$$

The optimization problem becomes:

$$\underset{\|\Delta W\|_2 \le \eta\sqrt{d_{out}/d_{in}}}{\operatorname{argmin}} \langle \nabla_W \mathcal{L}(W), \Delta W\rangle$$

The solution is a scaled version of the semi-orthogonal update:

$$\Delta W^* = -\eta\sqrt{\frac{d_{out}}{d_{in}}}U_r V_r^T$$

This scaling factor is precisely what is needed for width-aware updates, directly connecting the optimizer to the principles of MUP and ensuring stable learning dynamics as network width changes.

15.2 Key Idea 2: Efficient Orthogonalization with Newton-Schulz

The second key innovation of Muon is a computationally efficient method to approximate the semi-orthogonal matrix UV^T without performing an SVD. This is achieved via **Newton-Schulz iterations**.

 All Rights Reserved. This may not be publicly shared without explicit permission.

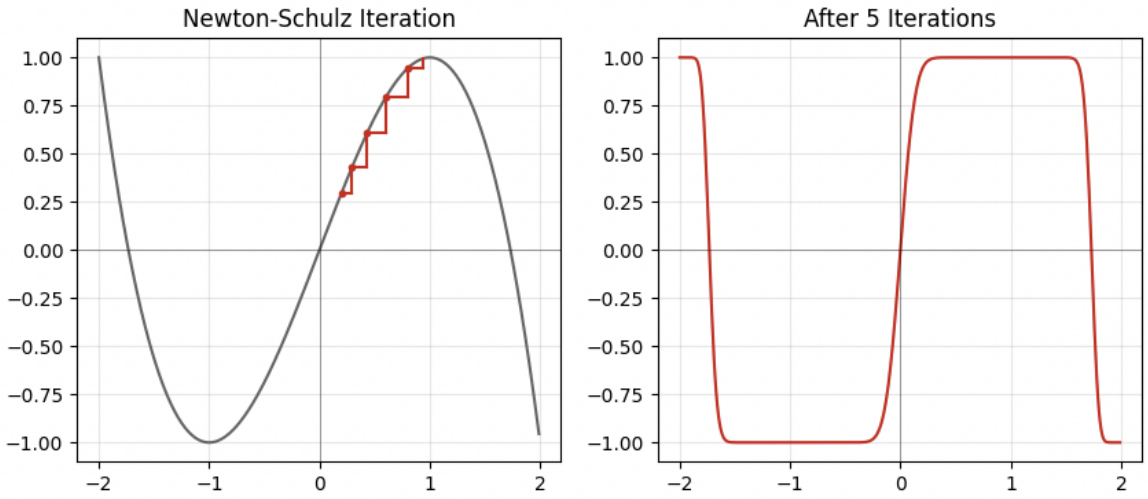


Figure 4: Newton-Schulz iteration maps all singular values of a matrix to $\text{sign}(\lambda)$. Shown above is the Newton-Schulz iteration for a scalar value.

The goal is to find a function f that can transform a matrix $G = U\Sigma V^T$ into its semi-orthogonal component UV^T. This is equivalent to finding a function that maps all singular values σ_i to 1. Muon achieves this via two observations:

(a) Getting the directions approximately correct is often sufficient. Perfect orthogonality is not required.

(b) An iterative method based on polynomial functions can efficiently approximate this transformation.

Odd Polynomials and the SVD: An important property is that odd polynomials "commute" with the SVD. For any odd polynomial $p(x)$ (e.g., $p(x) = a_1x + a_3x^3 + \dots$), applying it to a matrix $G = U\Sigma V^T$ results in:

$$p(G) = p(U\Sigma V^T) = Up(\Sigma)V^T$$

where $p(\Sigma)$ is the polynomial applied element-wise to the singular values on the diagonal. This means we can focus on finding a scalar polynomial $p(x)$ that, when applied iteratively, drives its input towards 1.

The Newton-Schulz Iteration: A standard polynomial used for this is:

$$p(x) = \frac{3}{2}x - \frac{1}{2}x^3$$

If we start with a value $x_0 \in (0, \sqrt{3})$ and repeatedly apply this function, $x_{k+1} = p(x_k)$, the sequence will converge to 1. Values in $(-\sqrt{3}, 0)$ converge to -1. Since singular values are always positive, we only care about the positive domain.

To ensure all singular values of our gradient matrix fall within this basin of convergence, we first normalize the gradient by its Frobenius norm: $\hat{G} = G/\|G\|_F$. Since $\|G\|_F \geq \|G\|_2 = \sigma_{max}$, all singular values of $\hat{G}$ will be in the interval $(0, 1]$, guaranteeing convergence.

15.3 The Full Muon Algorithm

Muon combines these ideas into a full optimizer algorithm:

 All Rights Reserved. This may not be publicly shared without explicit permission.

(a) **Momentum Accumulation:** Compute a momentum buffer B_t, similar to Adam or SGD with momentum.

$$B_t = \mu B_{t-1} + \nabla_W \mathcal{L}(W_t)$$

(b) **Orthogonalization:** Apply the Newton-Schulz iteration to the momentum buffer to get the semi-orthogonal update direction O_t. This involves:

- Normalizing: $\hat{B}_t = B_t / \|B_t\|_F$.
- Iteratively applying the polynomial $p(x)$ for a fixed number of steps (e.g., 5). Let the result be O_t.

(c) **Weight Update:** Update the weights using the orthogonalized direction, including the μP-inspired scaling factor.

$$W_{t+1} = W_t - \eta \cdot \sqrt{\frac{d_{out}}{d_{in}}} \cdot O_t$$

15.4 On Choosing the Polynomial

The choice of the polynomial $p(x)$ is a design parameter with important trade-offs.

- A higher-order polynomial might converge in fewer iterations but is more computationally expensive per iteration (requires more matrix multiplications).
- The polynomial used in the record-setting nanoGPT speedrun was a quintic:

$$p(x) = 3.444x - 4.7750x^3 + 2.0315x^5$$

Interestingly, for this polynomial, $p(1) \neq 1$. This means the iterations do not actually converge to a perfectly semi-orthogonal matrix. The rationale was that this polynomial gives a very steep slope near zero (large coefficient on the x term), which rapidly scales up small singular values. Empirically, this was found to accelerate training more than achieving perfect orthogonality, sacrificing theoretical convergence for practical speed.

15.5 Conclusion and Current Research

Muon is at the forefront of a new class of architecture-aware optimizers. This is an active area of research with several key takeaways:

- **Practical Considerations:** Muon's performance is sensitive to hyperparameters. For instance, it has been empirically observed to perform better with larger batch sizes, which lead to more stable gradient estimates.
- **Active Research:** The field is rapidly evolving with many variations and alternatives to Muon being proposed (e.g., Scion, Prawn), each with different trade-offs. Finding optimal polynomials is also an open research question.
- **Optimizer-Specific Tuning:** The rise of these complex optimizers underscores that hyperparameters are not universal. Stating that one optimizer is "better" than another requires careful, independent hyperparameter sweeps for both, as optimal settings for one may not be suitable for another.

 All Rights Reserved. This may not be publicly shared without explicit permission.